

Indian Institute of Technology
Department of Computer Science

CS202: Software Tools and Technologies Assessment 4

Jaskirat Singh Maskeen and Aarsh Wankar
(23110146, 23110003)

Instructor: Prof. Shouvick Mondal

November 23, 2025

Contents

11 Windows Forms App - Multi - Control Event Interaction	1
11.1 Introduction	1
11.2 Setup and Tools	1
11.3 Methodology, Execution and Results	2
11.3.1 Windows Forms App - Multi-Control Event Interaction:	2
11.3.2 Using EventArgs and Multiple Subscribers:	5
11.4 Output Reasoning Analyses	8
11.4.1 Output Reasoning (Level 0)	8
11.4.2 Output Reasoning (Level 1)	10
11.4.3 Output Reasoning (Level 2)	12
11.5 Conclusion	15
References	15
12 Adv. Event Handling & Custom EventArgs in C# Windows Forms	16
12.1 Introduction	16
12.2 Setup and Tools	16
12.3 Methodology	16
12.3.1 Creating the Form App	16
12.3.2 Custom Event Arguments	18
12.3.3 ProcessOrder Class	18
12.3.4 Event Chaining	19
12.3.5 Dynamic Subscription	19
12.3.6 Code	19
12.4 Results	22
12.4.1 User Interface and Input	22
12.4.2 Logic Validation	22
12.4.3 Successful Processing and Event Chaining	23
12.4.4 Dynamic Subscriber Management	24
12.5 Output Reasoning Analyses	25
12.5.1 Output Reasoning (Level 0)	25
12.5.2 Output Reasoning (Level 1)	27
12.5.3 Output Reasoning (Level 2)	30
12.6 Conclusion	34
References	34

Lab 11

Windows Forms App Multi Control Event Interaction

[Github Repository for Lab 11](#)

11.1 Introduction

In this lab, we deepen our understanding of event handling and delegates in C# Windows Forms Applications. We will learn how to create, subscribe, and invoke custom events to achieve modular, interactive, and reusable GUI designs using Visual Studio.

11.2 Setup and Tools

We use Visual Studio 2022 Community Version for .NET development and writing C# code along with .NET 8.0 (LTS) SDK. We use Windows 11 as our operating system. We first open a new Windows Forms Application project and name it 'EventsPlayground'. The screenshots are shown below:

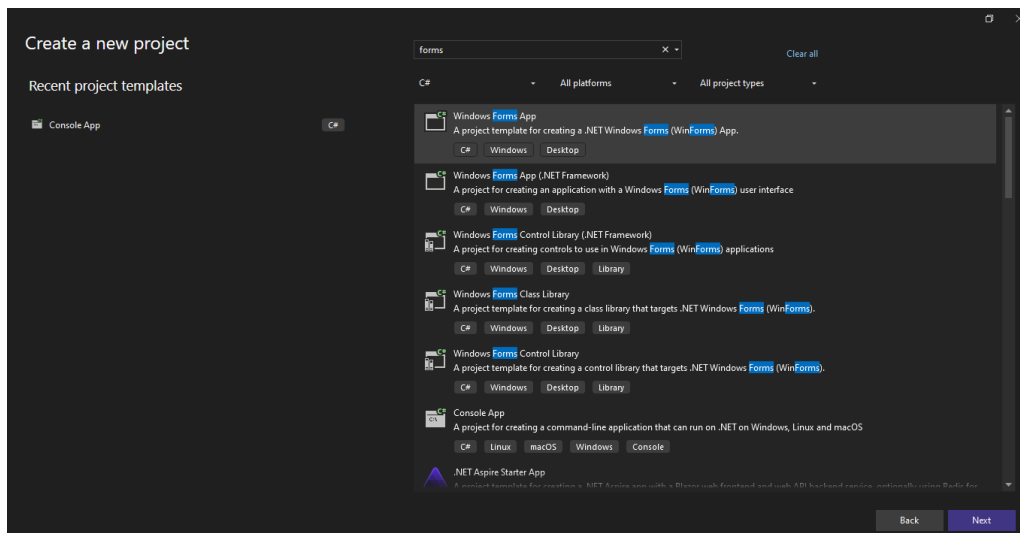


Figure 11.1: Creating a new project

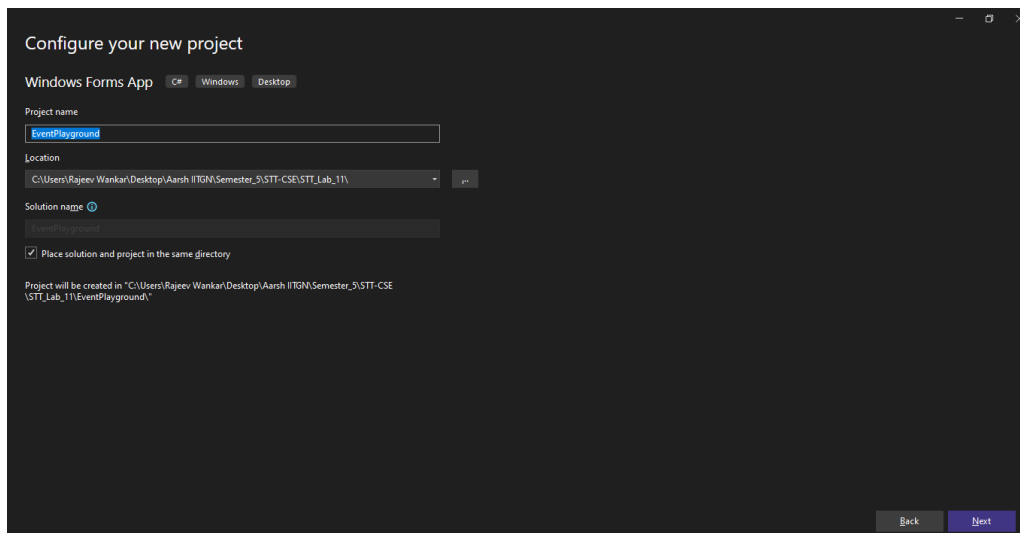


Figure 11.2: Naming the new project

11.3 Methodology, Execution and Results

11.3.1 Windows Forms App - Multi-Control Event Interaction:

We create a Windows Forms Application that makes use of delegates and events for handling button clicks. This application has two buttons and one text label. One of the buttons is for changing the color of text labels, and the other is for changing the text of the label. We define delegates and events, add the color and text changing logic to these events, and assign the buttons these events. Whenever these buttons are clicked, every method in the invocation list of these events is called, and the corresponding change is applied. The code is given below:

```

1 using System;
2 using System.Drawing;
3 using System.Windows.Forms;
4
5
6 namespace EventPlayground
7 {
8
9     // Custom Delegates
10
11     public delegate void ColorChangedEventHandler(Color color);
12     public delegate void TextChangedEventHandler(string text);
13
14
15     public class EventPlaygroundForm : Form
16     {
17
18         private Button btnChangeColor;
19         private Button btnChangeText;
20         private Label lblDisplay;
21         private ComboBox cmbColors;
22
23         private event ColorChangedEventHandler ColorChangedEvent;
24         private event TextChangedEventHandler TextChangedEvent;
25
26         public EventPlaygroundForm()
27         {
28
29             Text = "Event Playground";

```

```
30     Size = new Size(420, 200);
31     StartPosition = FormStartPosition.CenterScreen;
32     InitializeControls();
33     InitializeHandlers();
34 }
35
36 private void InitializeControls()
37 {
38     // Label
39     lblDisplay = new Label
40     {
41         Text = "Welcome to Events Lab",
42         AutoSize = false,
43         TextAlign = ContentAlignment.MiddleCenter,
44         Font = new Font("Segoe UI", 11f, FontStyle.Regular),
45         Size = new Size(360, 40),
46         Location = new Point(20, 15),
47         ForeColor = Color.Black,
48         BorderStyle = BorderStyle.FixedSingle
49     };
50     Controls.Add(lblDisplay);
51
52     // ComboBox
53     cmbColors = new ComboBox
54     {
55         Location = new Point(20, 70),
56         Size = new Size(120, 24),
57         DropDownStyle = ComboBoxStyle.DropDownList
58     };
59     cmbColors.Items.AddRange(new object[] { "Red", "Green", "Blue"
60 });
61     cmbColors.SelectedIndex = 0;
62     Controls.Add(cmbColors);
63
64     // Buttons
65     btnChangeColor = new Button
66     {
67         Text = "Change Color",
68         Location = new Point(160, 68),
69         Size = new Size(100, 28)
70     };
71     Controls.Add(btnChangeColor);
72
73     btnChangeText = new Button
74     {
75         Text = "Change Text",
76         Location = new Point(280, 68),
77         Size = new Size(100, 28)
78     };
79     Controls.Add(btnChangeText);
80
81 private void InitializeHandlers()
82 {
83     // Subscribe to events
84     ColorChangedEvent += OnColorChanged;
85     TextChangedEvent += OnTextChanged;
86     // Button Click Events
87     btnChangeColor.Click += (s, e) =>
88     {
```

```

89         Color selectedColor = cmbColors?.SelectedItem?.ToString()
90     switch
91     {
92         "Red" => Color.Red,
93         "Green" => Color.Green,
94         "Blue" => Color.Blue,
95         _ => Color.Black
96     };
97     ColorChangedEvent?.Invoke(selectedColor);
98     btnChangeText.Click += (s, e) =>
99     {
100         string newText = DateTime.Now.ToString("F");
101         TextChangedEvent?.Invoke(newText);
102     };
103 }
104
105 private void OnColorChanged(Color color)
106 {
107     lblDisplay.ForeColor = color;
108 }
109 private void OnTextChanged(string text)
110 {
111     lblDisplay.Text = text;
112 }
113
114 }
115
116 internal static class Program
117 {
118     /// <summary>
119     /// The main entry point for the application.
120     /// </summary>
121     [STAThread]
122     static void Main()
123     {
124         // To customize application configuration such as set high DPI
125         settings or default font,
126         // see https://aka.ms/applicationconfiguration.
127         ApplicationConfiguration.Initialize();
128         Application.Run(new EventPlaygroundForm());
129     }
130 }
131 }

```

The outputs are shown below:

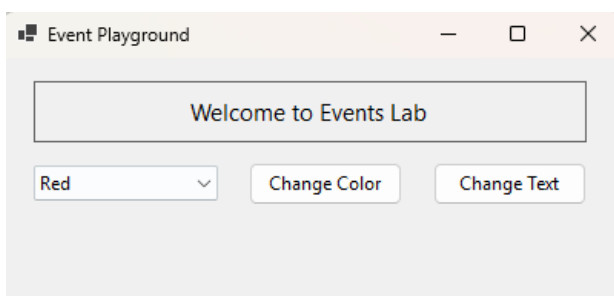


Figure 11.3: Output of the program

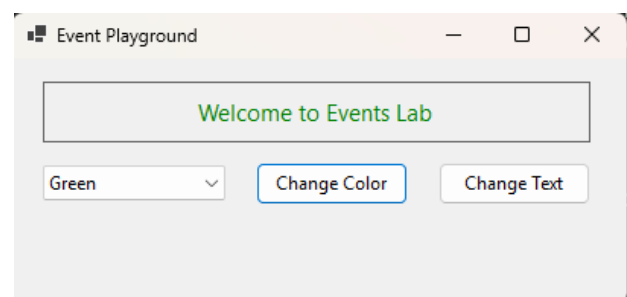


Figure 11.4: Output after pressing the Change Colour button

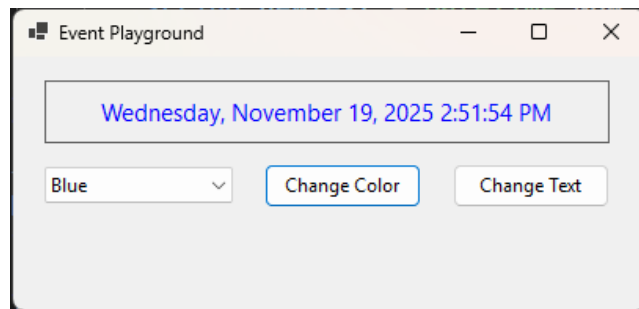


Figure 11.5: Output after pressing the Change Text button

Now, let us add in the EventArgs class which can be passed in to the events

11.3.2 Using EventArgs and Multiple Subscribers:

We now extend the first task by using a new custom EventArgs class for passing input to events, and we also modify the colour event to pass this data to multiple subscribers. We present a new notification that displays the selected colour when the colour button is clicked. The code is shown below:

```

1 using System;
2 using System.Drawing;
3 using System.Windows.Forms;
4
5
6 namespace EventPlayground
7 {
8
9     // Custom EventArgs class
10    public class ColorChangedEventArgs : EventArgs
11    {
12        public Color NewColor { get; }
13        public ColorChangedEventArgs(Color newColor)
14        {
15            NewColor = newColor;
16        }
17    }
18
19    public class TextChangedEventArgs : EventArgs
20    {
21        public string NewText { get; }
22        public TextChangedEventArgs(string newText)
23        {
24            NewText = newText;
25        }
26    }
27
28    // Custom Delegates
29
30    public delegate void ColorChangedEventHandler(ColorChangedEventArgs e);
31    public delegate void TextChangedEventHandler(TextChangedEventArgs e);
32
33
34
35    public class EventPlaygroundForm : Form
36    {
37
38        private Button btnChangeColor;
39        private Button btnChangeText;
40        private Label lblDisplay;

```

```
41     private ComboBox cmbColors;
42
43     private event ColorChangedEventHandler ColorChangedEvent;
44     private event TextChangedEventHandler TextChangedEvent;
45
46     public EventPlaygroundForm()
47     {
48         Text = "Event Playground";
49         Size = new Size(420, 200);
50         StartPosition = FormStartPosition.CenterScreen;
51         InitializeControls();
52         InitializeHandlers();
53     }
54
55     private void InitializeControls()
56     {
57         // Label
58         lblDisplay = new Label
59         {
60             Text = "Welcome to Events Lab",
61             AutoSize = false,
62             TextAlign = ContentAlignment.MiddleCenter,
63             Font = new Font("Segoe UI", 11f, FontStyle.Regular),
64             Size = new Size(360, 40),
65             Location = new Point(20, 15),
66             ForeColor = Color.Black,
67             BorderStyle = BorderStyle.FixedSingle
68         };
69         Controls.Add(lblDisplay);
70
71         // ComboBox
72         cmbColors = new ComboBox
73         {
74             Location = new Point(20, 70),
75             Size = new Size(120, 24),
76             DropDownStyle = ComboBoxStyle.DropDownList
77         };
78         cmbColors.Items.AddRange(new object[] { "Red", "Green", "Blue"
79     });
80     cmbColors.SelectedIndex = 0;
81     Controls.Add(cmbColors);
82
83     // Buttons
84     btnChangeColor = new Button
85     {
86         Text = "Change Color",
87         Location = new Point(160, 68),
88         Size = new Size(100, 28)
89     };
90     Controls.Add(btnChangeColor);
91
92     btnChangeText = new Button
93     {
94         Text = "Change Text",
95         Location = new Point(280, 68),
96         Size = new Size(100, 28)
97     };
98     Controls.Add(btnChangeText);
99 }
```



```

100
101     private void InitializeHandlers()
102     {
103         // Subscribe to events
104         ColorChangedEvent += OnColorChanged;
105         ColorChangedEvent += NotifyColorChange;
106         TextChangedEvent += OnTextChanged;
107
108         // Button Click Events
109         btnChangeColor.Click += (s, e) =>
110         {
111             Color selectedColor = cmbColors?.SelectedItem?.ToString()
switch
112             {
113                 "Red" => Color.Red,
114                 "Green" => Color.Green,
115                 "Blue" => Color.Blue,
116                 _ => Color.Black
117             };
118
119             ColorChangedEvent?.Invoke(new ColorChangedEventArgs(
selectedColor));
120         };
121         btnChangeText.Click += (s, e) =>
122         {
123             string newText = DateTime.Now.ToString("F");
124             TextChangedEvent?.Invoke(new TextChangedEventArgs(newText));
125         };
126     }
127
128     private void OnColorChanged(ColorChangedEventArgs e)
129     {
130         lblDisplay.ForeColor = e.NewColor;
131     }
132     private void OnTextChanged(TextChangedEventArgs e)
133     {
134         lblDisplay.Text = e.NewText;
135     }
136     private void NotifyColorChange(ColorChangedEventArgs e)
137     {
138         MessageBox.Show($"The colour has changed to {e.NewColor.Name}",
"Colour Changed!", MessageBoxButtons.OK, MessageBoxIcon.Information);
139     }
140 }
141
142 internal static class Program
143 {
144     /// <summary>
145     /// The main entry point for the application.
146     /// </summary>
147     [STAThread]
148     static void Main()
149     {
150         // To customize application configuration such as set high DPI
settings or default font,
151         // see https://aka.ms/applicationconfiguration.
152         ApplicationConfiguration.Initialize();
153         Application.Run(new EventPlaygroundForm());
154     }
155 }

```

156 }

The output is shown below:

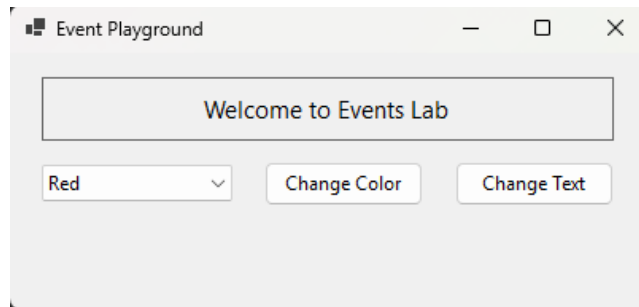


Figure 11.6: Starting screen of the program

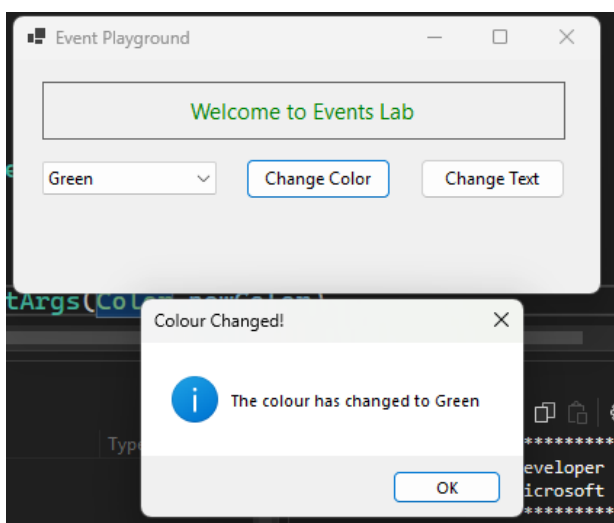


Figure 11.7: Notification called on colour change.

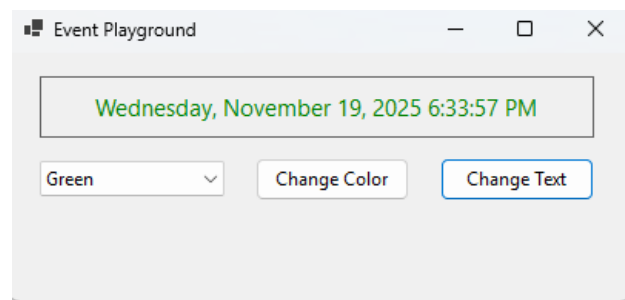


Figure 11.8: Text being changed normally.

11.4 Output Reasoning Analyses

11.4.1 Output Reasoning (Level 0)

We will now analyze the following C# programs and predict the outputs.

Program 1

```

1 using System;
2 delegate int Calc(int x, int y);
3 class Program
4 {
5     static int Add(int a, int b) { Console.Write("A"); return a + b; }
6     static int Mul(int a, int b) { Console.Write("M"); return a * b; }
7     static int Sub(int a, int b) { Console.Write("S"); return a - b; }
8     static void Main()
9     {
10         Calc c = Add;
11         c += Mul;
12         c += Sub;
13         c -= Add;
14         int res = c(2, 3);

```

```

15     Console.WriteLine(": " + res);
16 }
17 }

```

Listing 11.1: Program 1

In this code, the Calc c delegate first gets assigned the Add method, the Mul method, and finally the Sub method (in that order). However, with the -= command, the Add function gets removed from the invocation list of c. Hence, the only methods executed are Mul and Sub. Here is the predicted output:

MS:-1

It can be seen that the predicted output matches the true output:

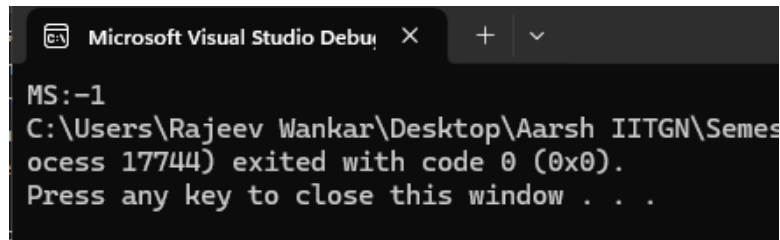


Figure 11.9: Output of Program 1

Program 2

```

1 using System;
2 delegate void ActionHandler(ref int x);
3 class Program
4 {
5     static void Inc(ref int a) { a += 2; Console.WriteLine("I" + a + " "); }
6     static void Dec(ref int a) { a--; Console.WriteLine("D" + a + " "); }
7     static void Main()
8     {
9         int val = 3;
10        ActionHandler act = Inc;
11        act += Dec;
12        act(ref val);
13        Console.WriteLine("F" + val);
14    }
15 }

```

Listing 11.2: Program 2

In this program, the delegate ActionHandler act stores two methods, Inc and Dec, both of which modify the same variable because it is passed by reference. The initial value is val = 3. When act(ref val) is invoked, Inc executes first, increasing val to 5 and printing I5. Then Dec runs on the updated value, decreasing val to 4 and printing D4. Finally, the statement Console.WriteLine("F" + val) prints F4, giving the overall output I5 D4 F4. Hence, the predicted output is:

I5 D4 F4

The true output matches our hunch:

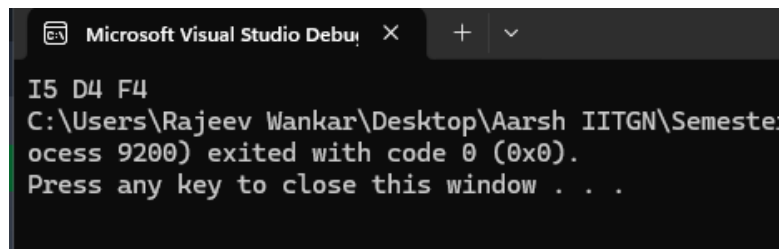


Figure 11.10: True output of Program 2

11.4.2 Output Reasoning (Level 1)

Program 3

```

1 using System;
2 class LimitEventArgs : EventArgs
3 {
4     public int CurrentValue { get; }
5     public LimitEventArgs(int val) => CurrentValue = val;
6 }
7 class Counter
8 {
9     public event EventHandler<LimitEventArgs> LimitReached;
10    public event EventHandler<LimitEventArgs> MilestoneReached;
11    private int value = 0;
12    public void Increment()
13    {
14        value++;
15        Console.WriteLine(">" + value);
16        // Fire Milestone event every 2nd increment
17        if (value % 2 == 0)
18            MilestoneReached?.Invoke(this, new LimitEventArgs(value));
19        // Fire Limit event every 3rd increment
20        if (value % 3 == 0)
21            LimitReached?.Invoke(this, new LimitEventArgs(value));
22    }
23 }
24 class Program
25 {
26     static void Main()
27     {
28         Counter c = new Counter();
29         // Subscribers for LimitReached
30         c.LimitReached += (s, e) => Console.WriteLine("[L" + e.CurrentValue + "]"
31         ");
32         c.LimitReached += (s, e) => Console.WriteLine("(Reset)");
33         // Subscribers for MilestoneReached
34         c.MilestoneReached += (s, e) =>
35         {
36             Console.WriteLine("[M" + e.CurrentValue + "]"
37             );
38             if (e.CurrentValue == 4)
39                 Console.WriteLine("{Alert}");
40         };
41         for (int i = 0; i < 6; i++)
42             c.Increment();
43     }
44 }

```

Listing 11.3: Program 3

In this program, each call to `Increment()` increases the counter's value, prints it, and then conditionally fires events: `MilestoneReached` on every even value and `LimitReached` on every multiple of three. The `MilestoneReached` event has one subscriber that prints `[Mvalue]` and additionally prints `Alert` when the value is 4, while `LimitReached` has two subscribers that print `[Lvalue]` and `(Reset)` in sequence. Executing the loop from 1 to 6 produces the following sequence: `>1` (no events), `>2 [M2]`, `>3 [L3] (Reset)`, `>4 [M4] Alert`, `>5` (no events), and `>6 [M6] [L6] (Reset)`, giving the final output `>1>2 [M2]>3 [L3] (Reset)>4 [M4] Alert>5>6 [M6] [L6] (Reset)`. The predicted output, therefore, is:

```
>1>2 [M2]>3 [L3] (Reset)>4 [M4] {Alert}>5>6 [M6] [L6] (Reset)
```

The screenshot of the output shown below conforms to our prediction.

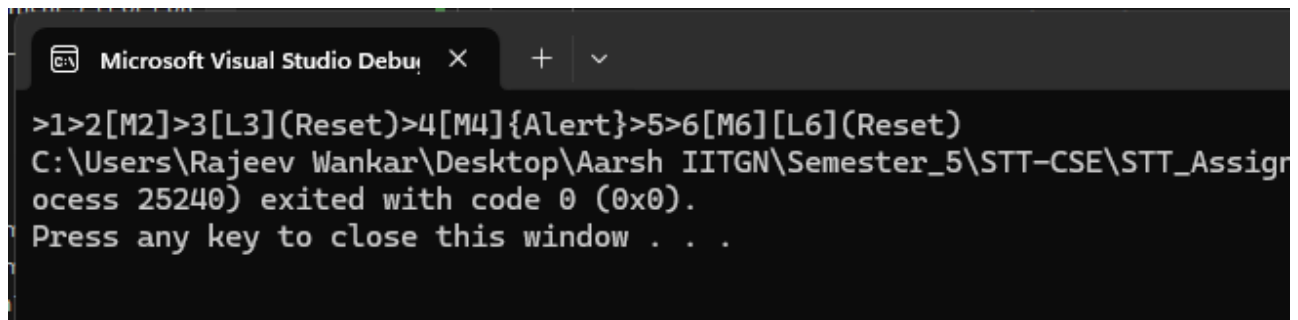


Figure 11.11: Output of Program 3

Program 4

```
1 using System;
2 using System.Numerics;
3 using System.Threading.Tasks;
4 using static System.Runtime.InteropServices.JavaScript.JSType;
5 class TemperatureEventArgs : EventArgs
6 {
7     public int OldTemperature { get; }
8     public int NewTemperature { get; }
9     public TemperatureEventArgs(int oldTemp, int newTemp)
10    {
11        OldTemperature = oldTemp;
12        NewTemperature = newTemp;
13    }
14 }
15 class TemperatureSensor
16 {
17     public event EventHandler<TemperatureEventArgs> TemperatureChanged;
18     private int temperature = 25;
19     public void UpdateTemperature(int newTemp)
20     {
21         int oldTemp = temperature;
22         temperature = newTemp;
23
24         if (Math.Abs(newTemp - oldTemp) > 5)
25         {
26             TemperatureChanged?.Invoke(this, new TemperatureEventArgs(
27                 oldTemp, newTemp));
28         }
29     }
30 }
31 class Program
32 {
33     static void Main()
```

```

33 {
34     TemperatureSensor sensor = new TemperatureSensor();
35     sensor.TemperatureChanged += (s, e) =>
36     Console.WriteLine($"Temperature changed from {e.OldTemperature}°C to
    {e.NewTemperature}°C");
37     sensor.TemperatureChanged += (s, e) =>
38     {
39         if (Math.Abs(e.NewTemperature - e.OldTemperature) > 10)
40             Console.WriteLine(" Warning: Sudden change detected!");
41     };
42     sensor.UpdateTemperature(28);
43     sensor.UpdateTemperature(30);
44     sensor.UpdateTemperature(46);
45     sensor.UpdateTemperature(52);
46 }
47 }

```

Listing 11.4: Program 4

In this program, the sensor starts at 25°C, and the TemperatureChanged event fires only when the temperature change exceeds 5 degrees. Two subscribers respond: the first always prints the old and new temperatures, while the second prints a warning only if the change is greater than 10 degrees. Calling UpdateTemperature(28) changes the temperature by 3, so no event fires; UpdateTemperature(30) changes it by 2, so again nothing happens; UpdateTemperature(46) changes it by 16, triggering both subscribers and producing "Temperature changed from 30°C to 46°C" followed by "Warning: Sudden change detected!"; finally, UpdateTemperature(52) changes it by 6, triggering only the first subscriber and printing "Temperature changed from 46°C to 52°C".

The predicted output, therefore, is:

```

Temperature changed from 30°C to 46°C
Warning: Sudden change detected!
Temperature changed from 46°C to 52°C

```

The prediction matches the true output:

```

Microsoft Visual Studio Debug Console
Temperature changed from 30°C to 46°C
Warning: Sudden change detected!
Temperature changed from 46°C to 52°C

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\STT-CSE\Process 15616) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 11.12: Output of Program 4

11.4.3 Output Reasoning (Level 2)

Program 5

```

1 using System;
2 class NotifyEventArgs : EventArgs
3 {
4     public string Message { get; }
5     public NotifyEventArgs(string msg) => Message = msg;
6 }

```

```

7 class Notifier
8 {
9     public event EventHandler<NotifyEventArgs> OnNotify;
10    public void Trigger(string msg)
11    {
12        Console.WriteLine("[Start]");
13        OnNotify?.Invoke(this, new NotifyEventArgs(msg));
14        Console.WriteLine("[End]");
15    }
16 }
17 class Program
18 {
19     static void Main()
20     {
21         Notifier n = new Notifier();
22         n.OnNotify += (s, e) =>
23         {
24             Console.WriteLine("{ " + e.Message + " }");
25         };
26         n.OnNotify += (s, e) =>
27         {
28             Console.WriteLine("(Nested)");
29             if (e.Message == "Ping")
30                 ((Notifier)s).Trigger("Pong");
31         };
32         n.Trigger("Ping");
33     }
34 }

```

Listing 11.5: Program 5

In this program, calling `n.Trigger("Ping")` prints `[Start]` and then invokes both subscribers in order. The first handler prints `Ping`, and the second prints `(Nested)` and, because the message is `"Ping"`, calls `Trigger("Pong")` again from inside the handler. This nested trigger prints another `[Start]`, then the handlers run again, printing `Pong(Nested)`, and because the message is now `"Pong"`, no further recursive trigger occurs, so `[End]` is printed. Control then returns to the outer trigger, which finally prints its own `[End]`, producing the full output: `[Start]Ping(Nested) [Start]Pong(Nested) [End] [End]`.

Hence, the expected output is:

```
[Start]{Ping}(Nested)[Start]{Pong}(Nested)[End][End]
```

The following screenshot of the output verifies this.

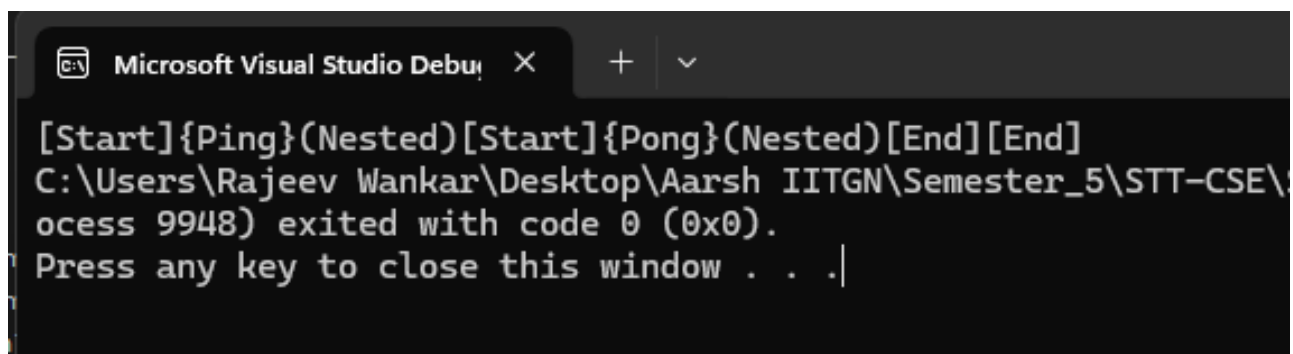


Figure 11.13: Output of program 5

Program 6

```

1 using System;
2 class AlertEventArgs : EventArgs
3 {
4     public string Info { get; }
5     public AlertEventArgs(string info) => Info = info;
6 }
7 class Sensor
8 {
9     public event EventHandler<AlertEventArgs> ThresholdReached;
10    public void Check(int value)
11    {
12        Console.WriteLine("[Check]");
13        if (value > 50)
14            ThresholdReached?.Invoke(this, new AlertEventArgs("High"));
15        Console.WriteLine("[Done]");
16    }
17 }
18 class Program
19 {
20     static void Main()
21     {
22         Sensor s = new Sensor();
23         s.ThresholdReached += (sender, e) =>
24         {
25             Console.WriteLine("{ " + e.Info + " }");
26             if (e.Info == "High")
27                 ((Sensor)sender).Check(30);
28         };
29         s.ThresholdReached += (sender, e) =>
30         {
31             Console.WriteLine("(Alert)");
32             s.Check(80);
33         }
34     }
35 }

```

Listing 11.6: Program 6

In this program, calling `s.Check(80)` prints `[Check]` and since the value is greater than 50, the `ThresholdReached` event fires, invoking its two handlers in order. The first handler prints `High` and, because the info is "High", it makes a nested call `Check(30)`, which prints `[Check]` again but does not trigger the event because 30 is not above the threshold, so it immediately prints `[Done]` and returns. After the first handler finishes, the second handler prints `(Alert)`, and finally the outer `Check(80)` call prints `[Done]`. Thus, the final output is:

```
[Check]{High}[Check][Done](Alert)[Done]
```

The following screenshot of the final output matches our prediction.

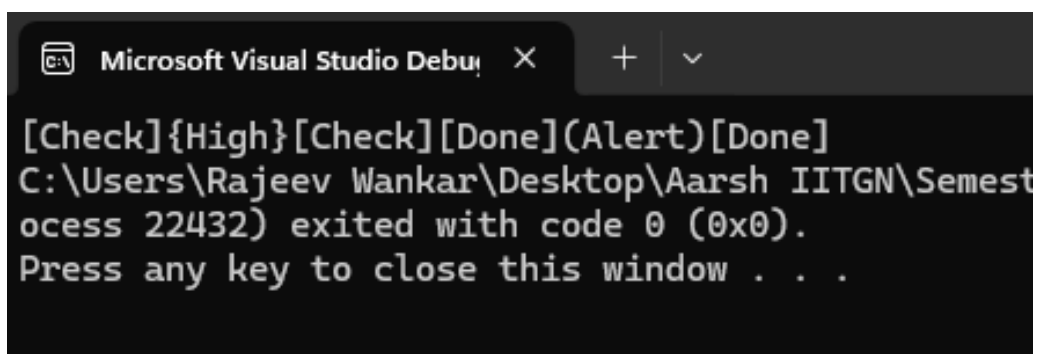


Figure 11.14: Output of program 6

11.5 Conclusion

This lab increased our understanding of the concepts of C# programming and the process of developing Windows Forms Applications within the .NET framework. We wrote and tested multiple programs, which helped us understand how Events and Delegates work in C#.

References

- [1] "Lab assignment document," Google Classroom, accessed: 2025-11-18.
- [2] Microsoft Learn, "Events (C# Programming Guide)," <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/events/>, accessed: 2025-11-19.
- [3] —, "Delegates overview (C#)," <https://learn.microsoft.com/en-us/dotnet/csharp/delegates-overview>, accessed: 2025-11-19.
- [4] —, "Windows Forms events (.NET)," <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/forms/events?tabs=dotnet>, accessed: 2025-11-19.
- [5] W. contributors. (2025) Event-driven programming. Accessed: 2025-11-19. [Online]. Available: https://en.wikipedia.org/wiki/Event-driven_programming

Lab 12

Advanced Event Handling and Custom EventArgs in C# Windows Forms Applications

[Github Repository for Lab 12](#)

12.1 Introduction

In this lab, we explored advanced event-driven programming paradigms within the .NET framework using C# Windows Forms. The main objective is to create a software system where business logic is separated from the user interface through the Publisher-Subscriber pattern. We develop an application named “Order Pipeline” that simulates a simplified e-commerce workflow. This lab focuses on implementing custom EventArgs for contextual data transmission, event chaining to model dependent workflows (such as validation following creation), and dynamic delegates to handle conditional features like express shipping notifications.

12.2 Setup and Tools

We use Visual Studio 2026 Community Version for .NET development and writing C# code along with .NET 10.0 SDK. We use Windows 11 as our operating system. We have already set up the .NET Development Environment, in the previous lab.

12.3 Methodology

The solution is designed around a separation of concerns between the User Interface (Form1) and the Business Logic (ProcessOrder). The communication between these layers is handled through events.

12.3.1 Creating the Form App

We first start a new project and name it “OrderPipeline” (Fig. [12.1](#)).

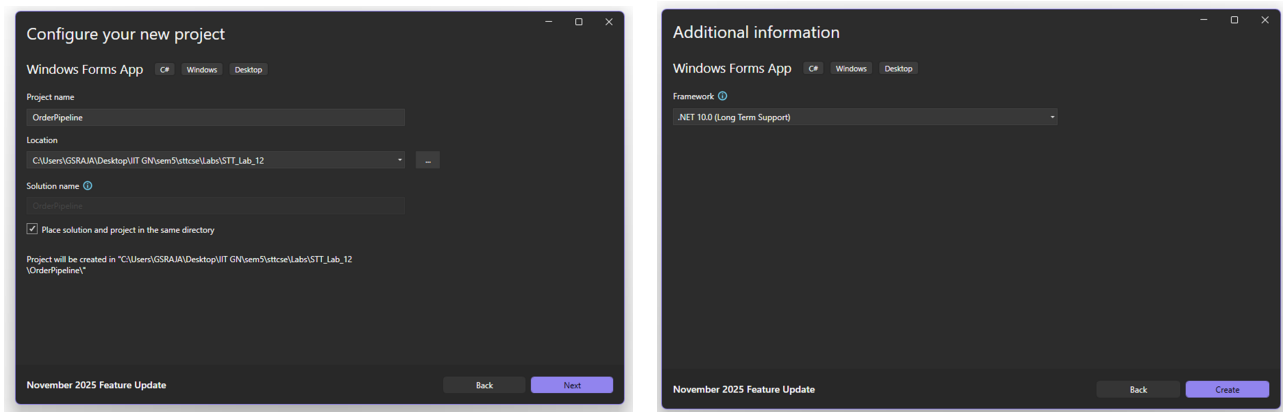


Figure 12.1: Creating a new form app.

We now create the UI of the form using various components from the Toolbox, and label each of them accordingly. Figures 12.2 - 12.5 show the form UI creation process.

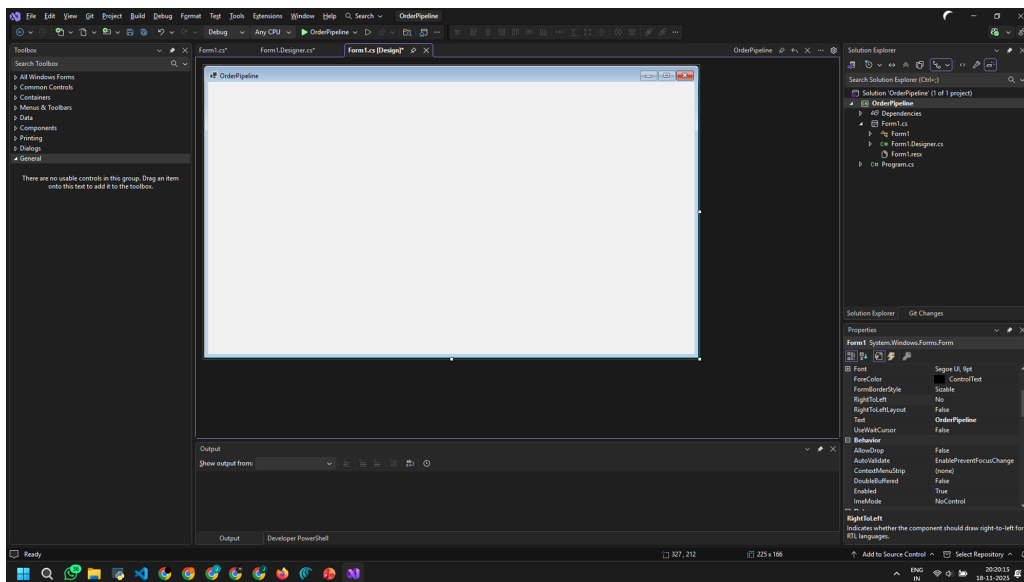


Figure 12.2: Creating the Form UI - Step 1

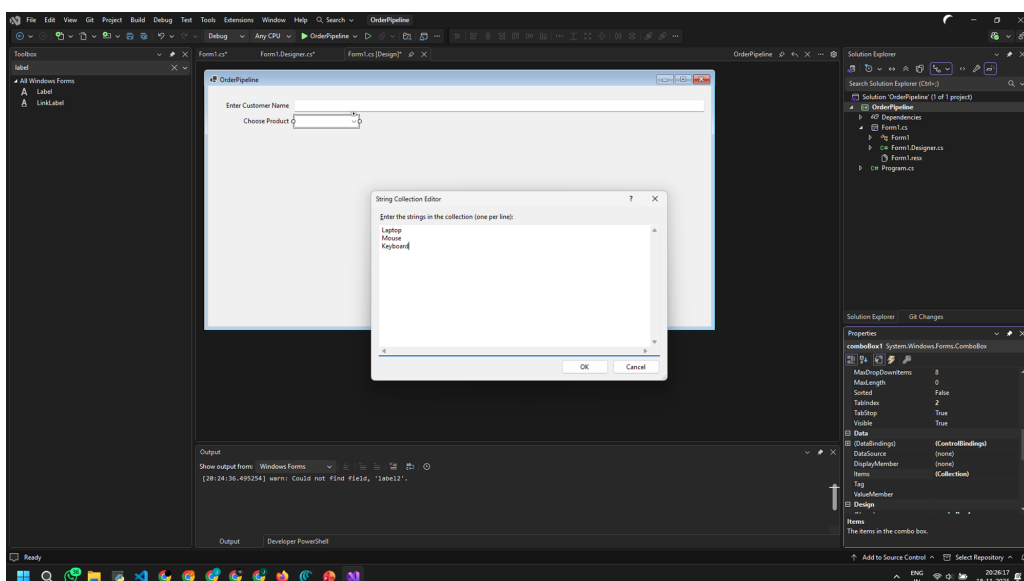


Figure 12.3: Creating the Form UI - Step 2

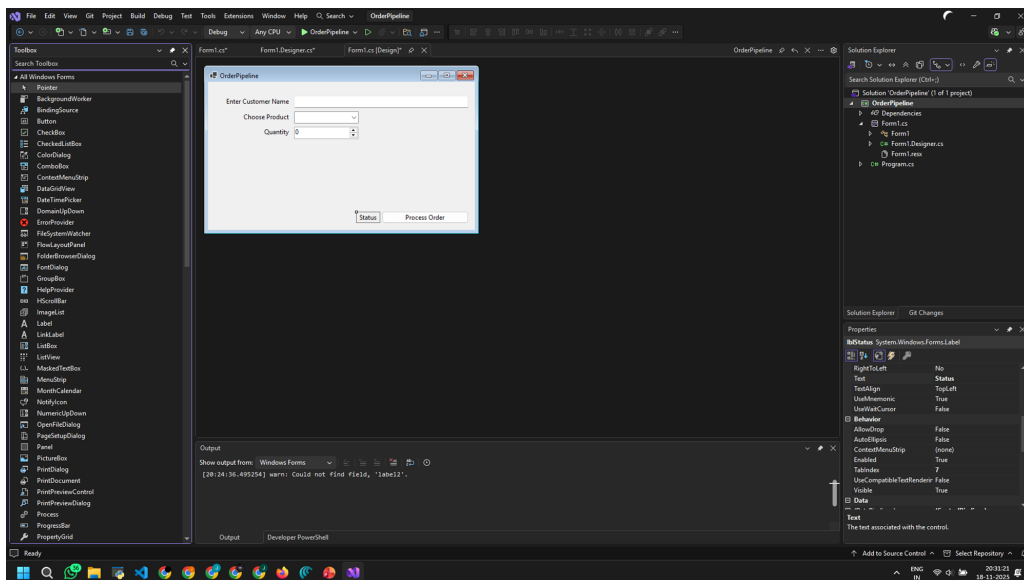


Figure 12.4: Creating the Form UI - Step 3

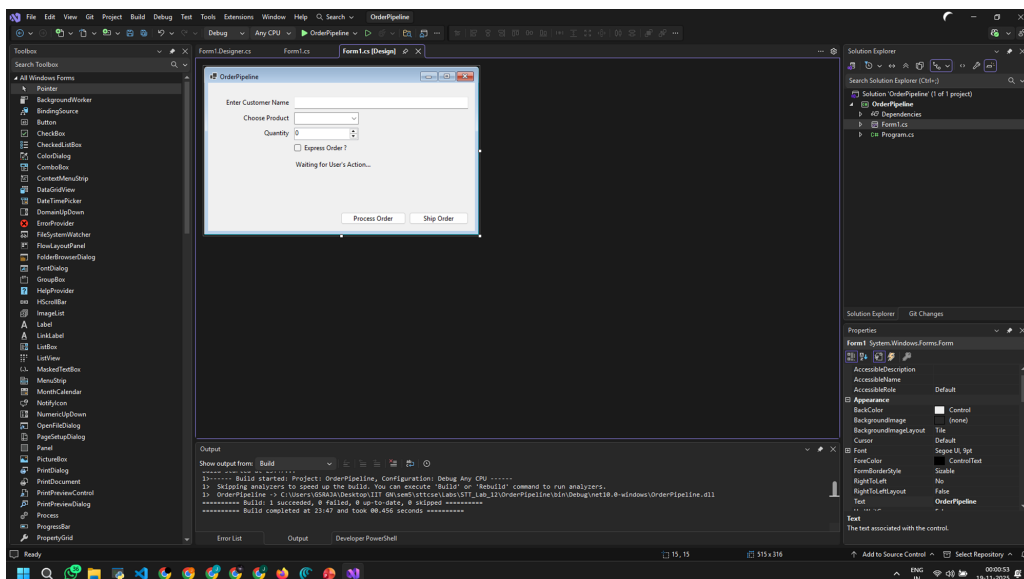


Figure 12.5: Creating the Form UI - Step 4

12.3.2 Custom Event Arguments

To transport specific data across the event pipeline, we implemented a custom class `ShipEventArgs` inheriting from `System.EventArgs`. This class encapsulates the Product name and the Express delivery status. This allows subscribers (such as the UI display methods) to access order details without referencing the logic class directly.

12.3.3 ProcessOrder Class

The `ProcessOrder` class acts as the Publisher. It defines four events:

- `OrderCreated`: Raised when the user initiates a process.
- `OrderConfirmed`: Raised internally if validation passes.
- `OrderRejected`: Raised internally if validation fails.
- `OrderShipped`: Raised when the product is dispatched.

12.3.4 Event Chaining

In the `ProcessOrder` constructor, we subscribe the internal method `ValidateOrder` to the `OrderCreated` event. Hence, when `CreateOrder()` is called:

1. The `OrderCreated` event executes, notifying the UI.
2. The chained `ValidateOrder` handler executes automatically.
3. Logic checks if `quantity > 0`.
4. Based on the result, it triggers the next event, either `OrderConfirmed` or `OrderRejected`.

12.3.5 Dynamic Subscription

To handle the “Express” feature, we used dynamic delegate subscription. In the `btnShipOrder_Click` event, we explicitly check the state of the `chkExpress` checkbox. If checked, we add (`+=`) the `NotifyCourier` handler to the `OrderShipped` event, otherwise, we ensure it is removed (`-=`). This shows how we can modify the object’s behavior by altering its invocation list. Also before checking the checkbox’s state, we unsubscribe from `NotifyCourier`, so that multiple clicks do not send multiple notifications.

12.3.6 Code

```

1 using System;
2 using System.Windows.Forms;
3
4 namespace OrderPipeline
5 {
6     public partial class Form1 : Form
7     {
8         ProcessOrder worker = new ProcessOrder();
9         public Form1()
10        {
11            InitializeComponent();
12            subscribe_events();
13
14        }
15        private void subscribe_events()
16        {
17            worker.OrderCreated += DisplayOrderInfo;
18            worker.OrderConfirmed += ShowConfirmation;
19            worker.OrderRejected += ShowRejection;
20            worker.OrderShipped += ShowDispatch;
21        }
22        private void DisplayOrderInfo(object sender, ShipEventArgs e)
23        {
24            MessageBox.Show($"Processing order for {e.Product}. Express: {e.
25Express}");
26        }
27        private void ShowConfirmation(object sender, ShipEventArgs e)
28        {
29            lblStatus.Text = $"Order Processed (and Validated) Successfully
30for {customer_name_textbox.Text}";
31        }
32        private void ShowRejection(object sender, EventArgs e)
33        {
34            lblStatus.Text = "Order Invalid - Please retry";
35        }
36        private void ShowDispatch(object sender, ShipEventArgs e)
37        {

```

```

36         lblStatus.Text = $"Product dispatched: [{e.Product}]";
37     }
38
39     private void NotifyCourier(object sender, ShipEventArgs e)
40     {
41         if (e.Express)
42         {
43             MessageBox.Show("Express delivery initiated!");
44         }
45     }
46
47     private void Form1_Load(object sender, EventArgs e)
48     {}
49
50     private void label1_Click(object sender, EventArgs e)
51     {}
52
53     private void comboBox1_SelectedIndexChanged(object sender, EventArgs
54     e)
55     {}
56
57     private void label3_Click(object sender, EventArgs e)
58     {}
59
60     private void btnProcessOrder_Click(object sender, EventArgs e)
61     {
62         string product = product_combobox.SelectedItem?.ToString() ?? "
Unknown";
63         int quantity = (int)qty_updown.Value;
64         bool is_order_express = chkExpress.Checked;
65
66         worker.CreateOrder(product, quantity, is_order_express);
67     }
68
69     private void customer_name_textbox_TextChanged(object sender,
EventArgs e)
70     {}
71
72     private void btnShipOrder_Click(object sender, EventArgs e)
73     {
74         string product = product_combobox.SelectedItem?.ToString() ?? "
Unknown";
75         bool is_order_express = chkExpress.Checked;
76         worker.OrderShipped -= NotifyCourier;
77         // Ensure no duplicate subscriptions so that multiple clicks
dont show multiple messages.
78         if (is_order_express)
79         {
80             worker.OrderShipped += NotifyCourier;
81         }
82         else
83         {
84             worker.OrderShipped -= NotifyCourier;
85         }
86         worker.ShipOrder(product, is_order_express);
87     }
88
89     private void qty_updown_ValueChanged(object sender, EventArgs e)
90     {}

```

```
91     }
92 }
93
94 public class ShipEventArgs : EventArgs
95 {
96     public string Product
97     {
98         get;
99     }
100     public bool Express
101     {
102         get;
103     }
104     public ShipEventArgs(string p, bool ex) => (Product, Express) = (p,
ex);
105 }
106
107 public class ProcessOrder
108 {
109     public delegate void ShipEventHandler(object sender, ShipEventArgs e
);
110     public delegate void OrderStatusHandler(object sender, EventArgs e);
111
112     public event ShipEventHandler OrderCreated;
113     public event ShipEventHandler OrderConfirmed;
114     public event OrderStatusHandler OrderRejected;
115     public event ShipEventHandler OrderShipped;
116
117     private bool is_order_cnf = false;
118     private int _qt = 0;
119
120     public ProcessOrder()
121     {
122         this.OrderCreated += ValidateOrder;
123     }
124
125     public void CreateOrder(string product, int quantity, bool express)
126     {
127         _qt = quantity;
128         ShipEventArgs args = new ShipEventArgs(product, express);
129         OrderCreated?.Invoke(this, args);
130     }
131
132     private void ValidateOrder(object sender, ShipEventArgs e)
133     {
134         if (_qt > 0)
135         {
136             is_order_cnf = true;
137             OrderConfirmed?.Invoke(this, e);
138         }
139         else
140         {
141             is_order_cnf = false;
142             OrderRejected?.Invoke(this, EventArgs.Empty);
143         }
144     }
145     public void ShipOrder(string product, bool express)
146     {
147         if (is_order_cnf)
148         {
```

```
149         ShipEventArgs args = new ShipEventArgs(product, express);
150         OrderShipped?.Invoke(this, args);
151     }
152     else
153     {
154         MessageBox.Show("Please process order first.");
155     }
156 }
157 }
158 }
```

Listing 12.1: Program 1*

12.4 Results

We have validated the OrderPipeline Form through a series of tests covering user input, logic validation, and dynamic event subscriptions.

12.4.1 User Interface and Input

The initial state of the application allows the user to select products from a predefined list. As shown in Figure 12.6, the ComboBox is populated with valid catalog items.

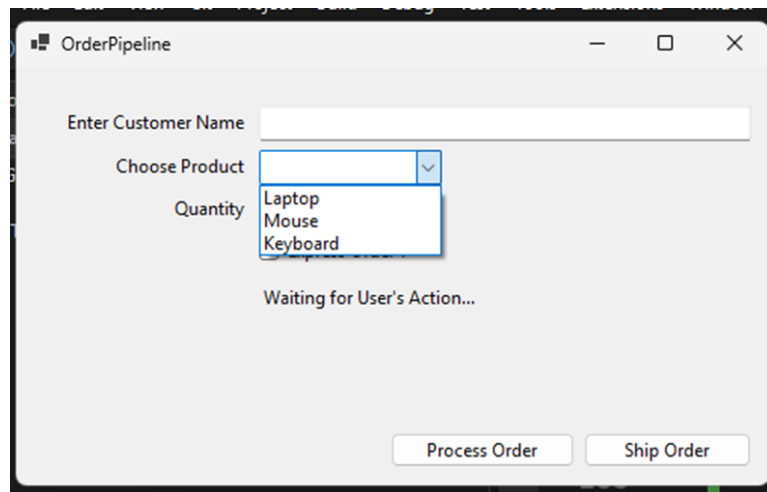


Figure 12.6: Initial UI State: Product Selection via ComboBox

12.4.2 Logic Validation

We test the following logical errors:

- **Premature Shipping:** When the user attempts to click “Ship Order” before processing, the system correctly identifies that the `is_order_cnf` flag is false and displays an error message (Figure 12.7 and Figure 12.8).
- **Quantity Validation:** When processing an order with a quantity of 0, the `ValidateOrder` subscriber detects the invalid state and raises the `OrderRejected` event. The status label updates to reflect this rejection (Figure 12.9).

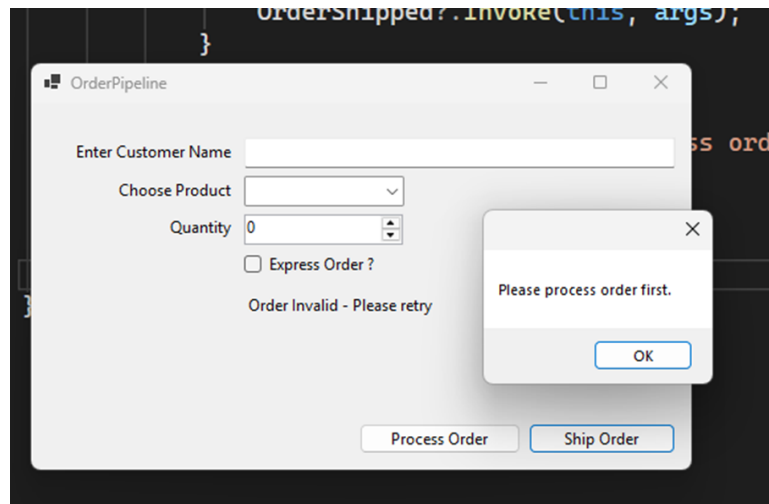


Figure 12.7: Workflow Logic: Error message when attempting to ship an unprocessed order

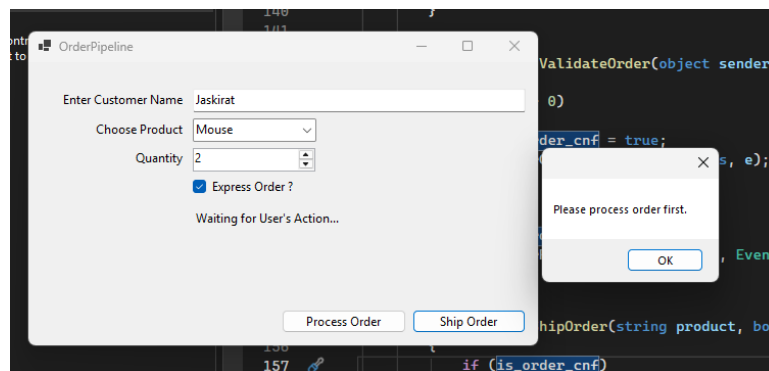


Figure 12.8: Workflow Logic: Error message when attempting to ship an unprocessed order (with express delivery)

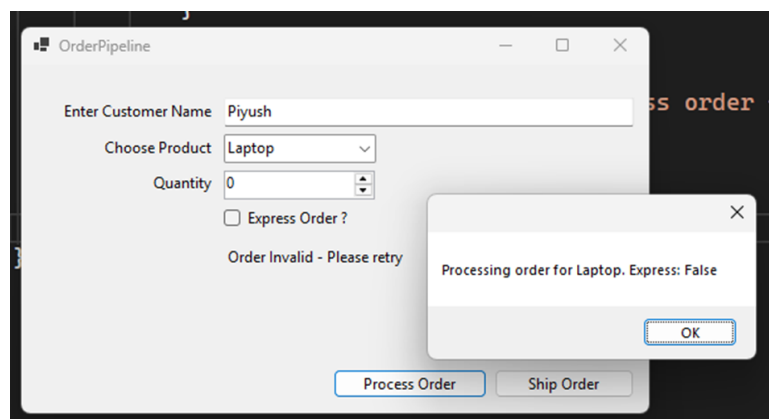


Figure 12.9: Event Chaining: Order Rejection triggered by invalid quantity (0)

12.4.3 Successful Processing and Event Chaining

When valid inputs are provided (e.g., Quantity > 0), the `OrderCreated` event successfully chains to the `OrderConfirmed` event. Figure 12.10 demonstrates the updated status label confirming the order for the customer.

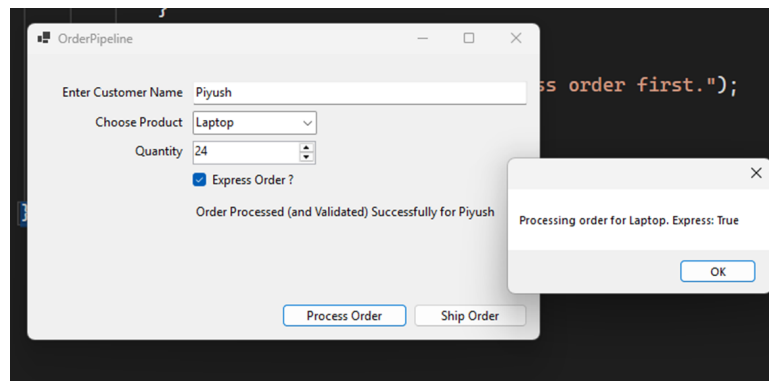


Figure 12.10: Success State: Valid order processed and confirmed via event chain

12.4.4 Dynamic Subscriber Management

The final test validates the dynamic subscription of the `NotifyCourier` delegate based on the “Express” checkbox.

- **Standard Shipping:** With the Express option unchecked, the product is dispatched, but the secondary courier notification does not appear (Figure 12.11). This confirms the handler was removed from the invocation list.
- **Express Shipping:** With the Express option checked, the `NotifyCourier` handler is dynamically added. As seen in Figure 12.12, the application displays both the dispatch status label and the additional “Express delivery initiated!” message box.

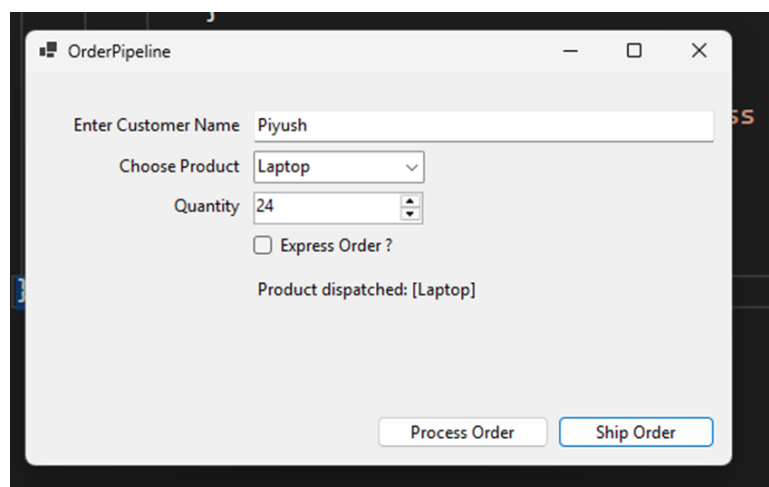


Figure 12.11: Dynamic Events: Standard shipping dispatch (No courier alert)

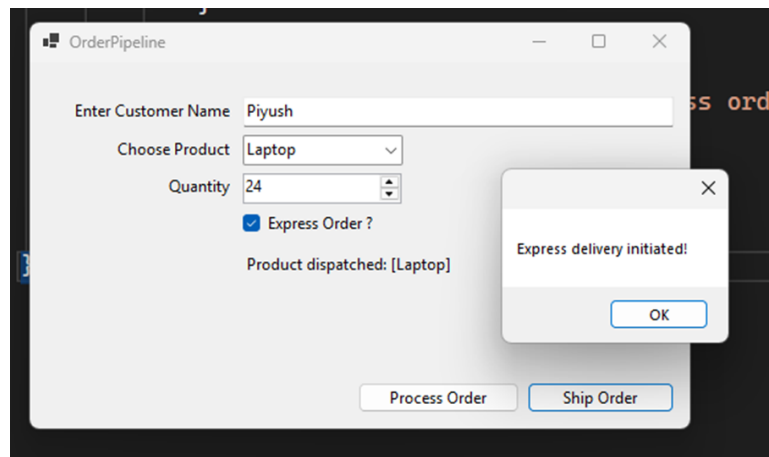


Figure 12.12: Dynamic Events: Express shipping triggering the multicast Courier Notification

12.5 Output Reasoning Analyses

12.5.1 Output Reasoning (Level 0)

Program 1

```

1 public delegate void AuthCallback(bool validUser);
2 public static AuthCallback loginCallback = Login;
3 public static void Login()
4 {
5     Console.WriteLine("Valid user!");
6 }
7 public static void Main(string[] args)
8 {
9     loginCallback(true);
10 }

```

Listing 12.2: Program 1

This function would not compile, because, methods and fields appear directly inside a namespace (CS0116), C# requires that fields, methods, and other executable statements must be contained inside a class or struct. Even if we fix this error by rewriting the above code as,

```

1 namespace MyConsoleApp
2 {
3     class Program
4     {
5         public delegate void AuthCallback(bool validUser);
6         public static AuthCallback loginCallback = Login;
7         public static void Login()
8         {
9             Console.WriteLine("Valid user!");
10        }
11        public static void Main(string[] args)
12        {
13            loginCallback(true);
14        }
15    }
16 }

```

Listing 12.3: Program 1*

Here, this would still not compile, as the delegate signature (`AuthCallback(bool validUser)`) does not match the method signature, `public static void Login()` (Fig. 12.13).

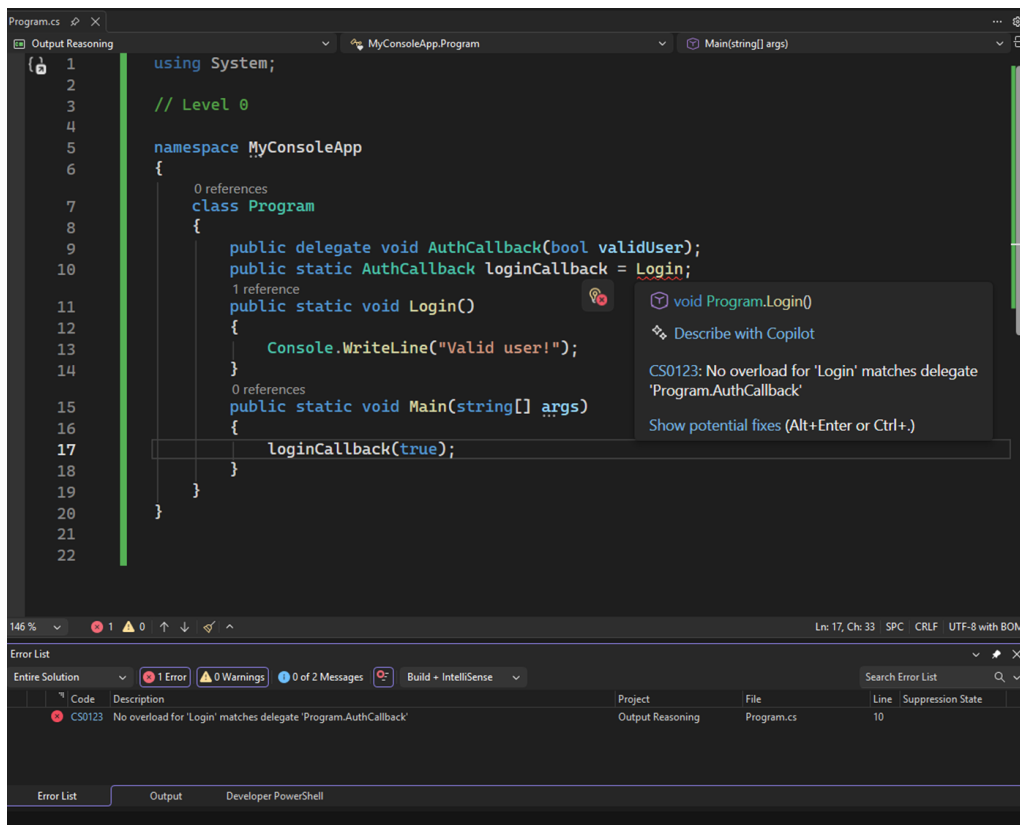


Figure 12.13: Program 1* fails to compile.

Program 2

```

1  using System;
2  delegate void Notify(string msg);
3  class Program
4  {
5      static void Main()
6      {
7          Notify handler = null;
8          handler += (m) => Console.WriteLine("A: " + m);
9          handler += (m) => Console.WriteLine("B: " + m.ToUpper());
10         handler("hello");
11         handler -= (m) => Console.WriteLine("A: " + m);
12         handler("world");
13     }
14 }

```

Listing 12.4: Program 2

The output will be:

```

A: hello
B: HELLO
A: world
B: WORLD

```

Following steps happen in the program to give the above output (Fig. 12.14):

1. At line 2, we declare the signature of the delegate, which returns nothing and takes a string as an input.
2. In the Main function, at line 7, we initialize the Notify handler to null.

3. Then we add two lambda functions, with the same signature to the delegate handler. Both lambda functions have the same signature, but one returns "A: " followed by the message, and the other returns "B: " followed by the uppercase transformed message.
4. At line 10, we call the handler, which runs both the above lambda functions in order.
5. We then remove a lambda function from the handler, which is exactly same as the first lambda function we added. However, delegates compare by reference, so removing this new lambda function, even though it is identical to the first one, does not do anything.
6. Therefore calling the handler on "world" will result in two lines as output "A: world", followed by "A: WORLD".

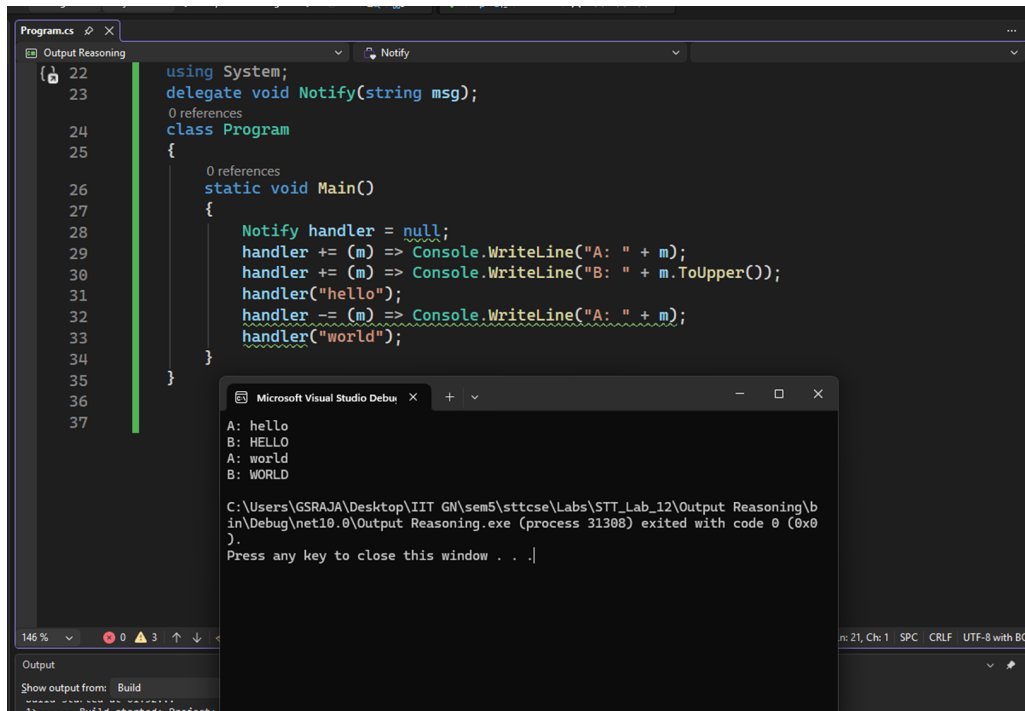


Figure 12.14: Program 2 Output

12.5.2 Output Reasoning (Level 1)

Program 3

```

1 using System;
2 class Program
3 {
4     static string txtAge;
5     static DateTime selectedDate;
6     static int parsedAge;
7     static void Main(string[] args)
8     {
9         try
10        {
11            Console.WriteLine(txtAge == null ? "txtAge is null" : txtAge);
12            Console.WriteLine(selectedDate == default(DateTime)
13            ? "selectedDate is default"
14            : selectedDate.ToString());
15            if (string.IsNullOrEmpty(txtAge))
16            {
17                Console.WriteLine("txtAge is null or empty, cannot parse");

```

```
18         }
19         else
20         {
21             parsedAge = int.Parse(txtAge);
22             Console.WriteLine($"Parsed Age: {parsedAge}");
23         }
24     }
25     catch (FormatException)
26     {
27         Console.WriteLine("Format Exception Caught");
28     }
29     catch (ArgumentNullException)
30     {
31         Console.WriteLine("ArgumentNullException Caught");
32     }
33     finally
34     {
35         Console.WriteLine("Finally block executed");
36     }
37 }
38 }
```

Listing 12.5: Program 3

The output will be (Fig. 12.15):

```
txtAge is null
selectedDate is default
txtAge is null or empty, cannot parse
Finally block executed
```

Following steps happen while running the above code:

1. txtAge, selectedDate, parsedAge are initialized to default values (null for string, 0 for int, and midnight Jan 1, 0001 for DateTime).
2. Since txtAge is null, "txtAge is null" is printed.
3. We never set the selected date, hence it contains its default value, which is midnight Jan 1, 0001. Hence "selectedDate is default" is printed. default(DateTime) returns the default value for DateTime.
4. Since, txtAge is null, string.IsNullOrEmpty(txtAge) is true, so "txtAge is null or empty, cannot parse" is printed.
5. Because of no exception, catch block doesn't run. Finally block always runs, so we get "Finally block executed" printed on the console.

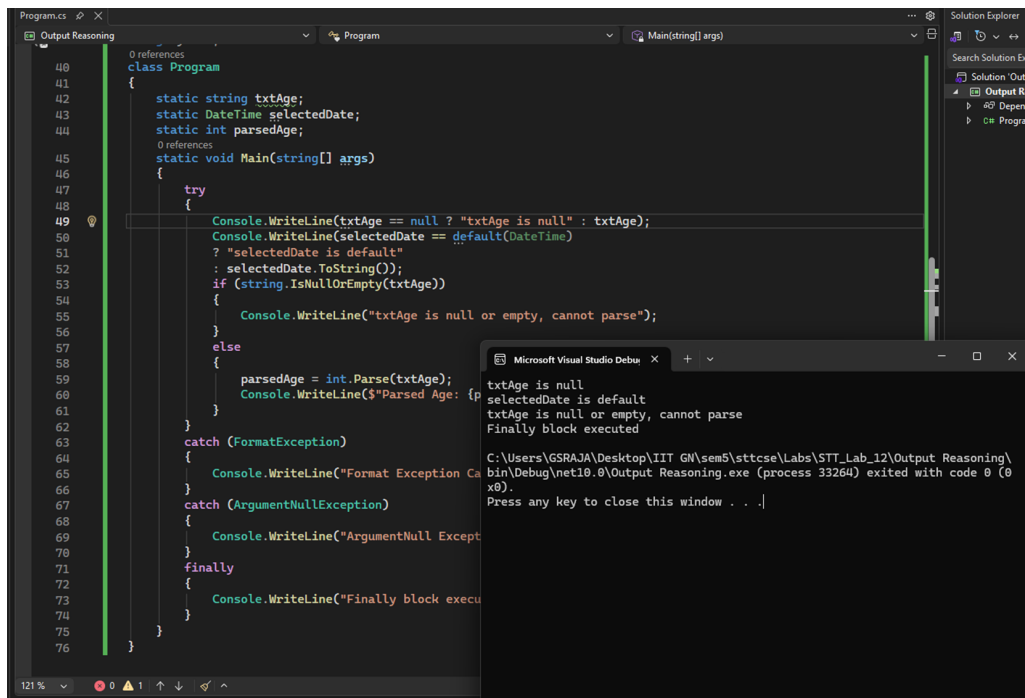


Figure 12.15: Program 3 Output

Program 4

```

1 using System;
2 delegate void Operation();
3 class Program
4 {
5     static void Main()
6     {
7         Operation ops = null;
8         ops += Step1;
9         ops += Step2;
10        ops += Step3;
11        try
12        {
13            ops();
14        }
15        catch (Exception ex)
16        {
17            Console.WriteLine("Caught: " + ex.Message);
18        }
19        Console.WriteLine("End of Main");
20    }
21    static void Step1()
22    {
23        Console.WriteLine("Step 1");
24    }
25    static void Step2()
26    {
27        Console.WriteLine("Step 2");
28        throw new InvalidOperationException("Step 2 failed!");
29    }
30    static void Step3()
31    {
32        Console.WriteLine("Step 3");
33    }

```

34 }

Listing 12.6: Program 4

The output will be (Fig. 12.16):

```
Step 1
Step 2
Caught: Step 2 failed!
End of Main
```

Following steps happen while running the above code:

1. A multicast delegate `ops` of type `Operation` is created and initialized to `null`.
2. Three methods (`Step1`, `Step2`, and `Step3`) are added to the invocation list of `ops` in that order.
3. The call `ops()` triggers execution of each method in the delegate's invocation list, sequentially.
4. `Step1` runs first and prints "Step 1".
5. Next, `Step2` runs. It prints "Step 2" and then throws an `InvalidOperationException` with the message "Step 2 failed!".
6. Once the exception is thrown inside the delegate chain, execution of remaining delegate methods (such as `Step3`) is skipped. The control jumps immediately to the nearest matching catch block.
7. The catch block catches the exception and prints "Caught: Step 2 failed!".
8. After the try-catch, program execution continues normally, and the last line "End of Main" is printed.

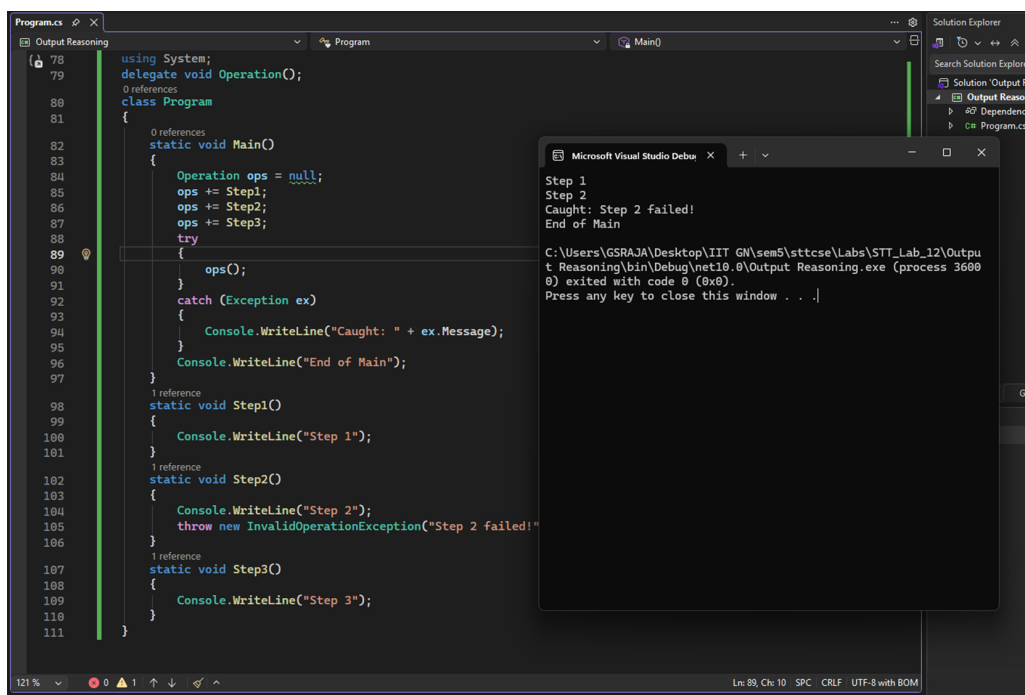


Figure 12.16: Program 4 Output

12.5.3 Output Reasoning (Level 2)

Program 5


```
1 using System;
2 namespace MethodOverloadingExample
3 {
4     class Program
5     {
6         static void Main(string[] args)
7         {
8             int x = 5;
9             new Base().F(x);
10            new Derived().F(x);
11            Console.ReadKey();
12        }
13    }
14    class Base
15    {
16        public void F(int x)
17        {
18            Console.WriteLine("Base.F(int)");
19        }
20    }
21    class Derived : Base
22    {
23        public void F(double x)
24        {
25            Console.WriteLine("Derived.F(double)");
26        }
27    }
28 }
```

Listing 12.7: Program 5

The output will be (Fig. 12.17):

```
Base.F(int)
Derived.F(double)
```

Following steps happen while running the above code:

1. In the Main function, a variable x is declared as an int and initialized to the value 5.
2. The call new Base().F(x) creates an instance of the Base class. Since x is an integer, it matches the method F(int) defined in Base. Therefore "Base.F(int)" is printed.
3. Next, the call new Derived().F(x) creates an instance of the Derived class. Even though x is still an integer, overload resolution is first performed within the Derived class.
4. The Derived class defines the method F(double). The integer value 5 can be implicitly converted to a double, so this overload is a valid candidate.
5. Overload resolution prefers methods declared in the most derived class when they are applicable. Therefore, the method Derived.F(double) is chosen, even though the argument is an integer.
6. Hence we get "Derived.F(double)" printed on the console.
7. Lastly, the program waits for a key press due to the call Console.ReadKey(). While running we press 'h' (Fig. 12.17).

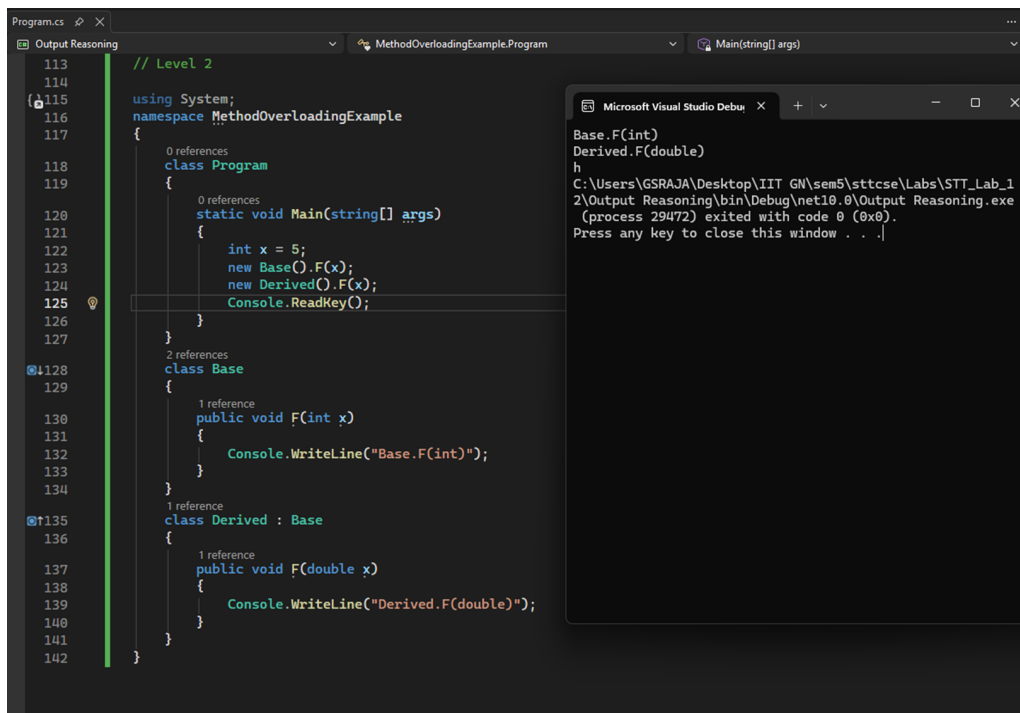


Figure 12.17: Program 5 Output

Program 6

```

1 using System;
2 class StepEventArgs : EventArgs
3 {
4     public int Step { get; }
5     public StepEventArgs(int s) => Step = s;
6 }
7 class Workflow
8 {
9     public event EventHandler<StepEventArgs> StepStarted;
10    public event EventHandler<StepEventArgs> StepCompleted;
11    public void Run()
12    {
13        for (int i = 1; i <= 3; i++)
14        {
15            StepStarted?.Invoke(this, new StepEventArgs(i));
16            Console.WriteLine($"[{i}]");
17            StepCompleted?.Invoke(this, new StepEventArgs(i));
18        }
19    }
20 }
21 class Program
22 {
23     static void Main()
24     {
25         Workflow wf = new Workflow();
26         wf.StepStarted += (s, e) =>
27         {
28             Console.WriteLine("<S" + e.Step + ">");
29             if (e.Step == 2)
30                 ((Workflow)s).StepCompleted += (snd, ev)
31                 => Console.WriteLine("(Dyn" + ev.Step + ")");
32         };
33         wf.StepCompleted += (s, e) => Console.WriteLine("<C" + e.Step + ">");

```

```
34         wf.Run();  
35     }  
36 }
```

Listing 12.8: Program 6

The output will be (Fig. 12.18):

<S1> [1] <C1> <S2> [2] <C2> (Dyn2) <S3> [3] <C3> (Dyn3)

Following steps happen while running the above code:

1. In the Main function, a Workflow object is created. Two event handlers are attached:
 - A StepStarted handler that prints <S{step}>. Additionally, when the step is 2, it dynamically adds another StepCompleted handler.
 - A StepCompleted handler that prints <C{step}>.
2. The method Run() iterates through steps 1, 2, and 3. For each iteration, the following sequence happens:
 - (a) Run StepStarted event.
 - (b) Print [i].
 - (c) Run StepCompleted event.
3. **Step 1:**
 - (a) StepStarted prints <S1>.
 - (b) [1] is printed by Run().
 - (c) StepCompleted prints <C1>.
 - (d) No dynamic handler is added, since this is not step 2.
4. **Step 2:**
 - (a) StepStarted prints <S2> and detects that the step is 2. It adds a new StepCompleted handler that prints (Dyn{step}), where step is the current step.
 - (b) [2] is printed.
 - (c) Two StepCompleted handlers now run:
 - i. The original one prints <C2>.
 - ii. The newly added dynamic handler prints (Dyn2).
5. **Step 3:**
 - (a) StepStarted prints <S3>. No new handlers are added.
 - (b) [3] is printed.
 - (c) Both StepCompleted handlers fire again:
 - i. The original handler prints <C3>.
 - ii. The dynamic handler (added during step 2) prints (Dyn3).

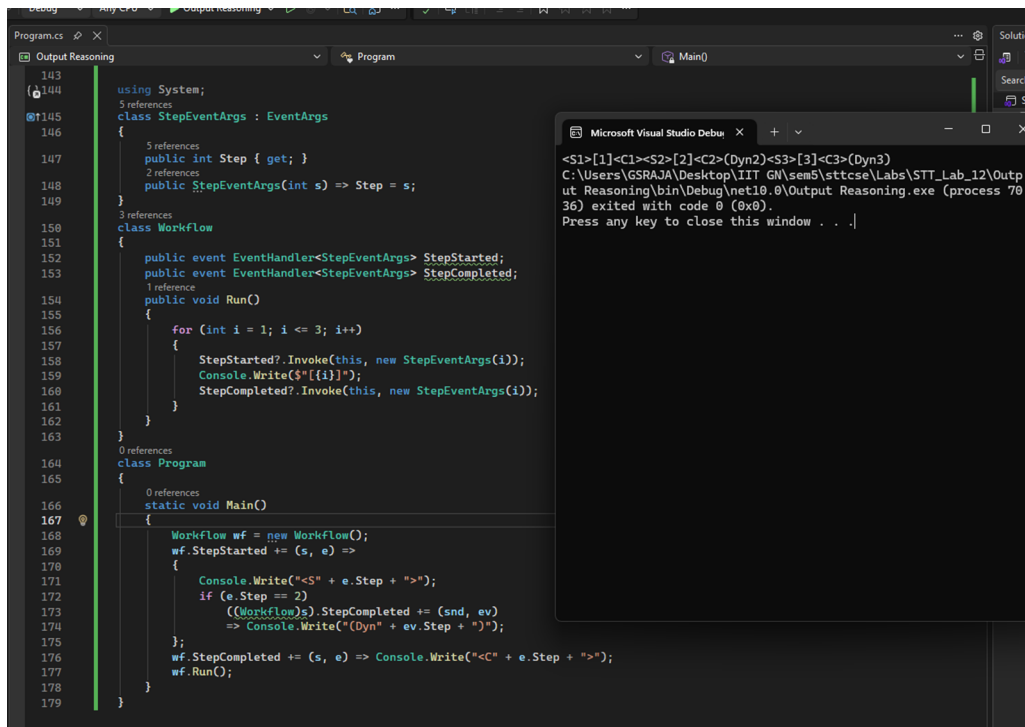


Figure 12.18: Program 6 Output

12.6 Conclusion

In this lab, we successfully demonstrated the practical application of advanced event-driven programming concepts in C#. Through the development of the `OrderPipeline` application, we established an architecture where the business logic is completely decoupled from the user interface. We used custom `EventArgs` to pass contextual data between components and utilized event chaining to enforce sequential business rules automatically. We also analyzed and reasoned through the behavior and output of various given programs.

References

- [1] Microsoft, "Constructors (c# programming guide)," <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constructors>, 2025, accessed: 2025-11-18.
- [2] "Lab assignment document," Google Classroom, accessed: 2025-11-18.
- [3] Microsoft, "Publisher-subscriber pattern (azure architecture center)," <https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber>, 2025, accessed: 2025-11-18.
- [4] Microsoft. (2022) Introduction to delegates and events – c#. Last updated: 31 March 2022. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/delegates-overview>
- [5] —. (2024) How to combine delegates (multicast delegates) – c# programming guide. Last updated: 22 December 2024. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/how-to-combine-delegates-multicast-delegates>